GUIA DO PROFESSOR

# Câmera Tapo C200/TC70: instalar, monitorar, gravar — e fazer a casa reagir

Plano de 6 aulas — instalação e posicionamento, app, armazenamento, RTSP/ONVIF, segurança de rede, ESP32 atuador e reconhecimento de gestos/rostos; ética e LGPD; gabarito e avaliação

---

**Câmera:** TP-Link Tapo C200 / TC70 • **Placa:** ESP32 Dev Module • **PC:** Python 3 + OpenCV 5 + MediaPipe 1.0

**Público:** Ensino Médio / Técnico (redes, eletrônica, informática) • **Carga horária:** 6 aulas de 100 min

Espaço Cria — 2026

# Sumário

1. Visão geral
2. Materiais
3. Preparação
4. Plano das aulas
5. Fundamentação para o professor
6. Erros comuns e como resolver
7. Gabarito e comentários dos desafios
8. Avaliação
9. Diferenciação
10. Ética, LGPD e acordo da turma
11. Referências

Anexo A — Scripts de referência

# 1. Visão geral

Esta sequência troca o "construir a câmera" (ESP32-CAM) por "**integrar um produto comercial**": instalar bem, configurar com critério, entender onde os dados vão parar e, por fim, abrir o produto pelos seus protocolos padrão (RTSP/ONVIF) para integrá-lo a um sistema próprio. É a competência mais pedida no mercado de automação e segurança: não inventar a câmera, mas saber **especificar, instalar, proteger e integrar**. As aulas 1–4 são tutoriais guiados com perguntas conceituais; as aulas 5–6 são projeto, reutilizando o ESP32 web, o OpenCV e o MediaPipe das sequências anteriores e acrescentando **reconhecimento facial** (OpenCV YuNet + SFace).

## Objetivos de aprendizagem

- Ler uma ficha técnica de câmera IP e relacionar cada item a física (sensor, lente, IR, codec) e a uso.
- Planejar a instalação (cobertura, altura, contraluz, Wi-Fi, energia) para um objetivo definido (laterais direita e fundos).
- Operar o app Tapo com critério: presets/patrulha, zonas de detecção, notificações, privacidade, compartilhamento.
- Comparar microSD, nuvem e NVR local (custo, retenção, risco, privacidade) e implementar um gravador RTSP.
- Explicar RTSP/ONVIF, NAT/P2P e as regras de segurança de dispositivos IoT (sem port-forwarding, 2FA, rede IoT, firmware).
- Construir um atuador HTTP com relé (isolamento, estado seguro, fallback local).
- Explicar detecção × reconhecimento facial, embeddings e limiar; implementar cadastro consentido e regras com cooldown.
- Argumentar sobre ética, consentimento e LGPD em videomonitoramento e reconhecimento facial.

## Competências (BNCC / Itinerários)

- **Física:** radiação infravermelha, óptica (campo de visão, distância focal), eletromagnetismo (relé).
- **Matemática:** trigonometria (cobertura), proporcionalidade (pixels por metro), taxas (bitrate × capacidade), vetores e similaridade de cosseno, matriz de confusão.
- **Redes e computação:** IP privado/NAT, portas e protocolos, codecs, API HTTP, threads, modelos de ML pré-treinados.
- **Cidadania digital:** LGPD, consentimento, vieses de reconhecimento facial, segurança de dispositivos conectados.

## Pré-requisitos

Sequências de LEDs (página web no ESP32), sensores (modo estação/mDNS ajuda) e ESP32-CAM (OpenCV, MediaPipe, NAT). Sem a de ESP32-CAM, acrescente 30 min na Aula 4 e 30 min na Aula 6 para OpenCV/MediaPipe.

# 2. Materiais

| Item | Qtd | Custo aprox. (R$) | Observação |
|---|---|---|---|
| Tapo C200 / TC70 + fonte | 1 por sala (ideal 1 por grupo nas aulas 4–6) | 200–300 | Qualquer versão; anote a versão de hardware |
| microSD 64–128 GB classe 10/U1 | 1 por câmera | 40–80 | Formatar no app |
| Suporte/parafusos, buchas, furadeira; extensão elétrica | — | 20 | Instalação física (professor) |
| ESP32 Dev Module + cabo | 1 por grupo | 35–60 | |
| Módulo relé 1 canal 5 V (KY-019 ou similar), LED amarelo + 220 Ω (lâmpada didática) | 1 | 8 | Ou lâmpada 12 V + fonte 12 V |
| Buzzer passivo, LED RGB + 3 × 220 Ω, botão, protoboard, jumpers | 1 kit | 15 | |
| PC com Python (OpenCV, MediaPipe, requests), webcam opcional | 1 por grupo | — | Para testes sem a câmera |
| Roteador Wi-Fi 2,4 GHz com acesso administrativo | 1 | — | Reserva de IP; rede de convidados/IoT |
| Cartaz “ambiente monitorado”; termo de consentimento | — | — | §10 |

## 3. Preparação

1. Crie a **ID TP-Link do laboratório** (e-mail institucional), ative 2FA, pareie a câmera e atualize o firmware antes da Aula 1. Guarde a senha da conta da câmera (RTSP) separada.
2. No roteador: reserve o IP da câmera (MAC) e, se possível, crie a rede de convidados/IoT. Confirme que **não** há isolamento de clientes (AP isolation) na rede usada pelos PCs/ESP32, senão o RTSP e o `luzes.local` não funcionam.
3. Teste `rtsp://user:senha@IP:554/stream1` no VLC e `python tapo_ver.py rtsp://…/stream2` em um PC do laboratório.
4. Ambiente Python: o mesmo das aulas anteriores + modelos em `aula-tapo/python/modelos/` (`gesture_recognizer.task`, `face_detection_yunet_2023mar.onnx`, `face_recognition_sface_2021dec.onnx`; o `setup-linux-mint.sh` baixa).
5. Compile e grave o sketch `01_esp32_luzes_alerta` com o Wi-Fi da sala; confira `http://luzes.local` no celular (Android recente e iOS resolvem mDNS; se não, use o IP).
6. Cadastre o próprio rosto como teste; apague depois. Leia o §10 (ética) e prepare o termo de consentimento.
7. Esquema Fritzing em `aula-tapo/esquemas/saida/01_esp32_rele_buzzer_rgb.fzz`; figura de topologia em `topologia_rede.png` (gerada por `topologia.py`).

# 4. Plano das aulas

## Aula 1 — Conhecendo e instalando

| Tempo | Atividade | Professor | Alunos | Evidência |
|---|---|---|---|---|
| 10' | Problema | "Precisamos vigiar a lateral direita e os fundos. Onde colocar UMA câmera?" Mostra a caixa e a ficha técnica | Propõem pontos na planta | Planta com marcações |
| 20' | Ficha técnica e física | Sensor/lente/campo (trigonometria no quadro), IR 850 nm (demonstra com câmera de celular que vê o IR), H.264 | Calculam cobertura a 5 m | Cálculo |
| 20' | Escolha do ponto | Critérios: altura, ângulo, contraluz, Wi-Fi (medir sinal), tomada; vota com a turma | Medem sinal, decidem | Ponto justificado |
| 20' | Instalação física | Professor fura/fixa; alunos conferem nível e inclinação | Auxiliam | Câmera fixada |
| 20' | Pareamento e config inicial | App, ID do lab, 2,4 GHz, firmware, fuso, inversão de imagem, LED | Acompanham no projetor; um grupo opera | Câmera on-line |
| 10' | Fechamento | Ticket: "por que a imagem noturna é P&B?" | Respondem | Ticket |

## Aula 2 — Monitorando pelo celular

| Tempo | Atividade | Professor | Alunos | Evidência |
|---|---|---|---|---|
| 15' | Local × internet | Desenha a topologia (figura); NAT e P2P; testa Wi-Fi × 4G ao vivo | Comparam latência | Medidas |
| 20' | Ao vivo e PTZ | Pan/tilt, presets Fundos/Lateral D/Porta, patrulha, áudio | Configuram (cada grupo um preset) | Presets salvos |
| 20' | Detecção e notificações | Zonas só na lateral/fundos; sensibilidade; programação; notificações ricas | Ajustam; testam alarmes falsos | Zero falsos em 10 min |
| 15' | Privacidade e compartilhamento | Modo privacidade (demonstra a lente girando), LED de status, compartilhar com visualização | Combinam a regra da sala | Regra escrita |
| 20' | Desafio 2.1 | Rotina horário × modo | Tabela + configuração | Tabela |
| 10' | Fechamento | "O que a câmera NÃO faz?" — ponte para o ESP32 | Listam | Lista |

## Aula 3 — Gravar e armazenar

| Tempo | Atividade | Professor | Alunos | Evidência |
|---|---|---|---|---|
| 15' | Três opções | Tabela comparativa; conta de capacidade (0,7 GB/h) | Calculam dias no cartão | Cálculo |
| 15' | microSD | Formatar no app; contínuo × eventos; loop; playback; download | Operam | Clipe baixado |
| 15' | Tapo Care | Teste grátis; onde ficam os dados; LGPD e transferência internacional | Debatem | — |
| 25' | NVR no PC | `tapo_gravar.py` contínuo e por movimento; `absdiff`; retenção | Rodam 20 min | Arquivos gerados |
| 20' | Desafios 3.1/3.2 | Política de retenção; pre-record com `deque` | Escrevem/programam | Política + script |
| 10' | Fechamento | Ticket: “por que gravar por evento economiza 10×?” | Respondem | Ticket |

## Aula 4 — RTSP, ONVIF e segurança

| Tempo | Atividade | Professor | Alunos | Evidência |
|---|---|---|---|---|
| 15' | Protocolos | RTSP 554 (stream1/2), ONVIF 2020, H.264; conta da câmera; IP reservado | Criam a conta; anotam IP | — |
| 15' | VLC | Abre `/stream1` e `/stream2`; latência; conexões simultâneas | Abrem nos PCs | Vídeo no PC |
| 20' | Python | `tapo_ver.py`; `CAP_FFMPEG`; TCP; a mesma matriz BGR | Rodam; foto e gravação | Foto salva |
| 20' | Segurança | Shodan (demonstração só de busca), por que não expor 554; 2FA; rede IoT; firmware; LED/privacidade | Checklist de segurança da sala | Checklist |
| 20' | Desafios 4.1/4.2 | Timelapse; ONVIF com `onvif-zeep` (preset por código) | Programam | Script |
| 10' | Fechamento | Ticket: “conta TP-Link × conta da câmera” | Respondem | Ticket |

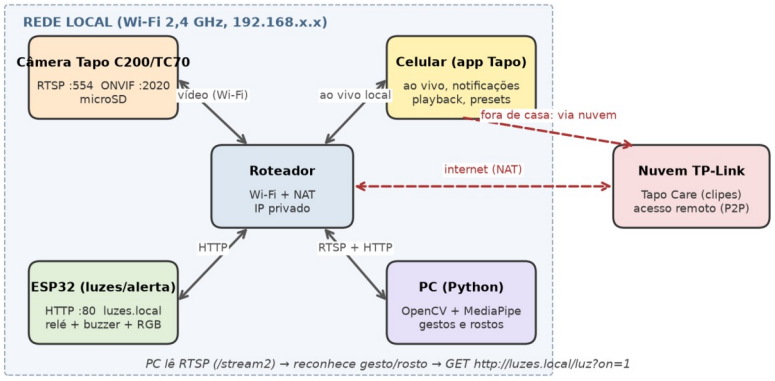



**Figura 1** — Topologia usada nas Aulas 2, 4 e 6.

## Aula 5 — ESP32 atuador

| Tempo | Atividade | Professor | Alunos | Evidência |
|---|---|---|---|---|
| 15' | Relé | Eletroímã + contato; isolamento; NO/NC/COM; ativo em LOW; **nunca 127/220 V sem o professor** | Identificam pinos | — |
| 20' | Montagem | Relé + LED-lâmpada + buzzer + RGB + botão (figura) | Montam | Checklist |
| 20' | Sketch 01 | Modo estação, mDNS, rotas, temporizador do alerta, botão local | Gravam; testam no celular | `luzes.local` operando |
| 25' | Desafios 5.1/5.3 | Temporizador; anti-spam | Programam | Sketch |
| 15' | Integração manual | Do PC: `curl`/navegador acionando; do app Tapo não dá — por quê? | Testam | — |
| 5' | Fechamento | Ticket: “estado seguro no boot” | Respondem | Ticket |

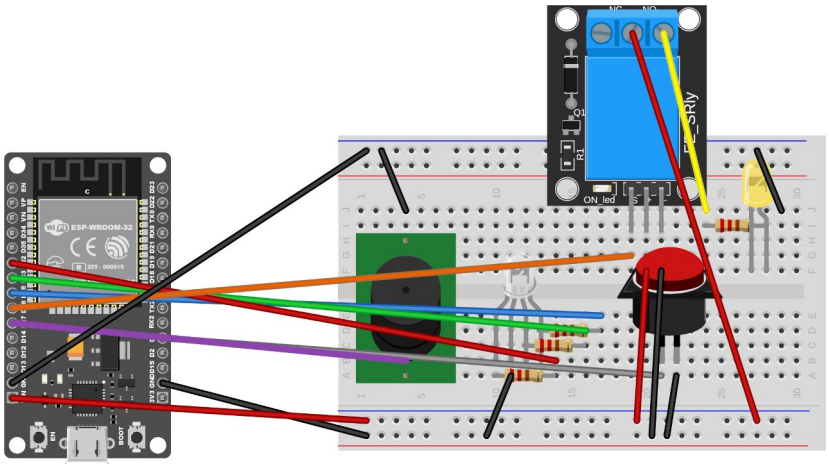


**Figura 2** — Aula 5 — montagem do atuador.

## Aula 6 — Projeto: gestos e rostos

| Tempo | Atividade | Professor | Alunos | Evidência |
|---|---|---|---|---|
| 15' | Detectar × reconhecer | YuNet, SFace, embedding de 128 números, cosseno, limiar 0,363; só números, não fotos | — | — |
| 15' | Consentimento e cadastro | Termo assinado; `cadastrar_rosto.py` (5–8 poses); `--listar/--apagar` | Cadastram 2–3 pessoas | Banco criado |
| 20' | Sistema completo | `tapo_gestos_faces.py` com a Tapo em `/stream2`; regras; cooldowns; threads | Rodam; entram na sala | Luz/alarme reagindo |
| 25' | Desafios 6.1/6.2 | Modo armado; tabela FP×FN por limiar (matriz de confusão) | Programam/medem | Tabela |
| 15' | Debate ético | Vieses, uso em escolas, LGPD; quando NÃO usar reconhecimento facial | Debatem | Síntese |
| 10' | Apresentação | Grupos demonstram; rubrica | Apresentam | Rubrica |

# 5. Fundamentação para o professor

## Hardware e física da C200/TC70

Sensor CMOS 1/2,9" 2 MP, lente fixa ~4 mm (≈65° H), processador com H.264 (1080p a 15 fps nas v1–v3; 30 fps nas v5+), WDR, LEDs IR 850 nm com filtro IR-cut mecânico (ouve-se o "clique" ao mudar de modo). O TC70 é a designação de mercado brasileiro da C200 (mesmo app, mesmo firmware). Pan 360° (com batente físico) e tilt 114° por motores de passo. Fonte 9 V/0,6 A (versões novas USB-C 5 V). Wi-Fi 2,4 GHz apenas — roteadores com *band steering* podem atrapalhar o pareamento: separe o SSID 2,4 GHz temporariamente.

## Cobertura e resolução

Largura de campo L = 2·d·tan(FOV/2): a 5 m com 65°, L ≈ 6,4 m; densidade ≈ 1920/6,4 ≈ 300 px/m. Para identificação facial recomenda-se ≥ 250 px/m (DORI: *detect* 25, *observe* 63, *recognize* 125, *identify* 250 px/m). Logo, reconhecer rostos é viável a ≤ 5–6 m em 1080p — por isso o projeto aponta a câmera para a porta. Instalação alta (2,2–2,6 m) reduz oclusão e vandalismo, mas aumenta a inclinação do rosto; 15–25° é o compromisso.

## Rede: P2P, RTSP, ONVIF

O app usa o serviço TP-Link para *hole punching* (P2P); quando falha, retransmite pela nuvem. RTSP: `rtsp://user:pass@IP:554/stream1` (1080p) e `/stream2` (360p), ambos H.264 + AAC; exige a "conta da câmera" (Avançado → Conta da câmera). ONVIF Profile S na porta 2020 (`/onvif/device_service`), com PTZ e presets — base dos desafios 4.2/6.6 (biblioteca `onvif-zeep`). A câmera aceita poucas sessões RTSP simultâneas (2–3); o ao vivo do app conta. Firmwares de 2023+ exigiram reativar ONVIF e recriar a conta após atualização — primeiro item do diagnóstico.

## Armazenamento e bitrate

1080p H.264 da Tapo fica em ~1–2 Mbit/s (0,45–0,9 GB/h); `/stream2` ~0,3 Mbit/s. microSD em FAT32/exFAT, loop quando cheio; versões antigas limitam a 128 GB. Tapo Care: clipes de eventos (não 24 h), 7/30 dias (180 em alguns mercados), teste grátis 30 dias, preços regionais no app. O gravador Python escreve MJPEG (AVI) por simplicidade — ~10× maior que H.264; para produção, usar `ffmpeg -c copy` (sem recodificar) ou um NVR (Frigate).

## Segurança

Câmeras IoT são o alvo nº 1 de botnets (Mirai): senhas padrão, portas expostas, firmware antigo. Regras: 2FA na ID TP-Link; conta da câmera com senha própria; nunca encaminhar 554/2020; rede IoT isolada (VLAN ou convidados) — mas lembre que o PC/ESP32 precisam alcançar a câmera, então ou ficam na mesma rede IoT, ou o roteador permite tráfego entre elas; firmware automático; desligar ONVIF/RTSP quando não usados. Demonstrar o Shodan apenas como busca (sem acessar dispositivos alheios).

## Relé e atuação

O KY-019 usa um transistor NPN + optoacoplador (em alguns) acionando um SRD-05VDC (10 A/250 VAC); o ESP32 fornece 3,3 V no sinal — suficiente para a maioria, mas alguns módulos de 5 V ficam "meio ligados": usar módulo de 3,3 V ou um transistor extra. Estado inicial: no boot os GPIOs ficam em alta impedância e o

relé pode acionar por instantes — `setLuz(false)` cedo e, se preciso, pull-up externo. Carga de 127/220 V só em caixa fechada, com disjuntor, pelo professor; na aula, LED/12 V.

### Reconhecimento facial

YuNet (2023): detector CNN de ~75 k parâmetros, 5 landmarks, ~5 ms em CPU. SFace (2021): rede treinada com perda de margem esférica; gera embedding de 128-D; limiar de cosseno 0,363 (ou L2 1,128) recomendado pelo OpenCV. Precisão típica > 99 % em LFW, mas cai com iluminação ruim, oclusão, ângulos > 30° e baixa resolução (< 60 px entre olhos). **Vieses** documentados em modelos faciais (NIST FRVT): taxas de erro maiores para alguns grupos demográficos — discutir obrigatoriamente. Guardar só embeddings reduz, mas não elimina, a sensibilidade do dado (continua biométrico pela LGPD: dado pessoal **sensível**, art. 5º II e art. 11). Em escolas, use apenas com consentimento explícito, finalidade didática e exclusão ao fim.

### Arquitetura do projeto

Câmera → RTSP (`/stream2`) → OpenCV decodifica → MediaPipe (gestos) + YuNet/SFace (rostos) → regras com histerese temporal (votação, `TEMPO_INTRUSO`, cooldowns) → HTTP para `luzes.local` em thread → ESP32 aciona e desarma sozinho. Latência total típica: 0,5–1 s (RTSP 0,3–0,5 + inferência 0,05 + HTTP 0,05). Pontos de falha e mitigação: Wi-Fi (TCP no RTSP, reconexão), câmera ocupada (fechar o app), mDNS (usar IP), PC lento (`/stream2`, pular quadros).

## 6. Erros comuns e como resolver

| Sintoma | Causa provável | Solução |
|---|---|---|
| Câmera não pareia | Rede 5 GHz/band steering; senha errada; celular longe | SSID 2,4 GHz dedicado; perto do roteador; RESET 5 s |
| LED vermelho fixo / 'off-line' | Wi-Fi fraco no ponto; fonte fraca | Medir sinal; repetidor; fonte original |
| Imagem invertida | Instalada de cabeça para baixo | App → Configurações → Inverter imagem |
| Notificações falsas | Ventilador, luz, cortina; sensibilidade alta; zona inteira | Zonas; sensibilidade média; horário |
| Visão noturna 'branca' / reflexo | Vidro ou objeto perto refletindo IR | Afastar do vidro; não filmar através de janela |
| microSD não reconhecido | > limite da versão; formatação; cartão falso | Formatar no app; 64–128 GB de marca |
| RTSP 401/404 ou 'não abriu' | Conta da câmera não criada; URL errada; sessões esgotadas; AP isolation | Criar conta; `/stream1`; fechar app; roteador |
| Vídeo picotado no OpenCV | UDP em Wi-Fi fraco | `rtsp_transport;tcp` (já no script) |
| `luzes.local` não resolve | mDNS bloqueado/isolado; Windows antigo | Usar o IP; instalar Bonjour; AP isolation |
| ESP32 não entra na rede | SSID/senha; rede 5 GHz; portal cativo | 2,4 GHz; conferir; rede IoT |
| Relé não aciona / aciona invertido | Módulo de 5 V com sinal de 3,3 V; lógica ativa-baixa | Módulo 3,3 V ou transistor; `RELE_ATIVO_BAIXO` |

| Sintoma | Causa provável | Solução |
|---|---|---|
| Relé clica no boot | GPIO flutuando | `setLuz(false)` cedo; pull-up; GPIO sem função de strapping |
| Rosto conhecido dá 'desconhecido' | Luz ruim; ângulo; poucas amostras; muito longe | Cadastrar 5–8 poses; aproximar; `/stream1` |
| Desconhecido aceito como conhecido | Limiar baixo; amostras ruins | Subir `LIMIAR_COS` para 0,40–0,45; recadastrar |
| Gestos não reconhecidos na Tapo | Mão pequena a 360p; contraluz | `/stream1`; aproximar; luz frontal |
| Python lento / atrasado | Decodificar 1080p + 2 modelos | `/stream2`; processar 1 a cada 2 quadros |

# 7. Gabarito e comentários dos desafios

## Aula 1

- **1.1** Com três presets o canto cobre tudo exceto a área sob a própria câmera ("ponto cego" de ~1,5 m): uma segunda câmera ficaria no canto oposto.
- **1.2** 300 px/m a 3 m → rosto de 15 cm ≈ 45 px (limite); a 8 m ≈ 17 px (só detecção). Reconhecimento confiável ≤ 5 m.
- **1.3** O preto fosco absorve o IR e não o reflete para a câmera; o branco reflete — a visão noturna depende da **refletância no IR**, que nem sempre coincide com a cor visível (alguns tecidos pretos refletem IR).

## Aula 2

- **2.1** Exemplo: aula (7–17 h) = privacidade ligada; intervalo = patrulha sem notificação; noite/fim de semana = detecção nas zonas + notificação + gravação por evento.
- **2.2** Menos pixels monitorados = menos fontes de movimento irrelevante (corredor, porta de outra sala); também reduz gravação por evento.
- **2.3** Tempo dominado pela nuvem (push) — 3–10 s; o processamento local da câmera é < 1 s.

## Aula 3

- **3.1** Avaliar: finalidade, base legal, retenção (ex.: 7 dias), acesso (professor + coordenação), descarte (loop + apagar nuvem), avisos.
- **3.2** `buffer = deque(maxlen=900)`; ao detectar movimento, `for q in buffer: gravador.write(q)` antes de seguir.
- **3.3** `if not (7 <= datetime.now().hour < 17)`.
- **3.4** microSD: ~R$ 80 uma vez; Tapo Care: ~R$ 20 × 36 = R$ 720; PC 15 W × 24 h × 365 × 3 ≈ 394 kWh ≈ R$ 350 + hardware. Para a escola: microSD + PC já existente para integração.

## Aula 4

- **4.1** `if time.time() - ultima >= 60: cv2.imwrite(...)`; depois `VideoWriter` a 10 fps.
- **4.2** `from onvif import ONVIFCamera; cam = ONVIFCamera(ip, 2020, user, pwd); ptz = cam.create_ptz_service(); ptz.GotoPreset({'ProfileToken': token, 'PresetToken': '1'})`.
- **4.3** `subprocess.run(['ping','-c','1',ip])` ou socket na 554; Telegram `sendMessage`.
- **4.4** Tapo: barato por qualidade, estável, app, nuvem, fechado; ESP32-CAM: aberto, programável, barato, instável, MJPEG, sem PTZ. Produto final → Tapo; aprender/integrar sensores exóticos → ESP32-CAM.

## Aula 5

- **5.1** `desligarEm = millis() + min*60000`; no `loop`, `if (desligarEm && millis() > desligarEm) setLuz(false)`; botão zera `desligarEm`.
- **5.2** Tabela `cenas[] = {{"aula",1,0},{"vazia",0,0},{"noite",0,1}}`.

- **5.3** Vetor circular dos últimos 5 instantes; se `millis() - t[0] < 60000`, ignorar e piscar amarelo.
- **5.4** `prefs.putBool("luz", luzLigada)` em `setLuz`; `carregar` no `setup`.
- **5.5** `configTime` + vetor de 10 `String`; rota `/` monta a lista.

### Aula 6

- **6.1** `armado` alterna no gesto; `if ha_desconhecido and armado`. LED vermelho via `/led?cor=vermelho`.
- **6.2** Matriz de confusão por limiar: baixo (0,30) → mais FP (desconhecido aceito), alto (0,45) → mais FN (conhecido rejeitado). Laboratório: 0,40 (segurança); cofre: 0,45 + segundo fator.
- **6.3** Dois `enviar()` com hosts diferentes; o braço precisa estar em modo estação na mesma rede.
- **6.4** `csv.writer` com `datetime.now()`; `pandas` + `matplotlib` para o histograma.
- **6.5** `requests.post(f"https://api.telegram.org/bot{T}/sendPhoto", data={"chat_id": id}, files={"photo": open(foto,"rb")})`.
- **6.6** ONVIF `GotoPreset` + aguardar `GetStatus` MoveStatus = IDLE antes de processar rostos.

## 8. Avaliação

### Rubrica

| Critério | Iniciante (1) | Em desenvolvimento (2) | Proficiente (3) |
|---|---|---|---|
| Instalação e configuração | Câmera on-line | Ponto justificado; presets e zonas corretas | Cobertura calculada; rotina horário × modo; sem falsos alarmes |
| Armazenamento e redes | Grava no cartão | Compara 3 opções; RTSP no VLC/Python | NVR com retenção/pre-record; explica NAT/P2P/portas; checklist de segurança aplicado |
| Atuador ESP32 | Relé liga pela página | API completa; botão local; alerta temporizado | Cenas/temporizador/anti-spam; estado seguro justificado |
| Visão e IA | Scripts rodam | Explica detecção × reconhecimento, embedding, limiar | Mede FP×FN e escolhe limiar; regras com cooldown; integra 2 dispositivos |
| Ética e privacidade | Segue regras | Aplica consentimento e retenção | Argumenta sobre vieses/LGPD e propõe salvaguardas |
| Comunicação | Um aluno faz tudo | Papéis definidos | Demonstração completa e clara da cadeia câmera → PC → ESP32 |

### Formativa

- Tickets por aula; planta com cones de visão (Aula 1); tabela de rotina (Aula 2); política de retenção (Aula 3); checklist de segurança (Aula 4); tabela FP×FN (Aula 6); termo de consentimento assinado.

## 9. Diferenciação

- **Avançados:** ONVIF por código (4.2/6.6), Frigate/Home Assistant, Telegram a partir do Python, segundo ESP32.
- **Dificuldade:** aulas 1–4 são operacionais e inclusivas; na 5–6, usar `--webcam --sem-esp` e o ESP32 de demonstração; pares.
- **Sem Tapo para todos:** uma câmera por sala; grupos revezam o RTSP (limite de sessões!) ou usam webcam/ESP32-CAM como fonte com o mesmo script.
- **Sem internet:** Aula 3 sem Tapo Care (demonstração em vídeo); tudo o mais é local.

## 10. Ética, LGPD e acordo da turma

- Imagem e biometria facial são dados pessoais (biometria = **sensível**). Base legal na escola: consentimento específico e informado (ou interesse legítimo só para segurança, sem biometria).
- Regras mínimas: finalidade escrita; cartaz de aviso; retenção definida; acesso restrito; exclusão ao fim (microSD, nuvem, PC, banco de rostos); nenhuma publicação.
- Reconhecimento facial: **só** com consentimento individual, para a atividade didática, apagando o banco (`--apagar`) ao final; nunca para controle de frequência ou disciplina sem política institucional.
- Modelo de termo: “Eu, ___, autorizo o uso da minha imagem e do cadastro facial (vetores numéricos) exclusivamente na atividade didática ___ em __/__/__, com exclusão ao término. Posso revogar a qualquer momento.”
- Debate da Aula 6: vieses (NIST FRVT), falsos positivos em segurança pública, direito à privacidade em espaços de ensino.

## 11. Referências


- TP-Link — Tapo C200: página do produto, datasheet e FAQ (RTSP/ONVIF, Tapo Care, microSD): https://www.tp-link.com/br/ e https://www.tapo.com/
- TP-Link FAQ — ver câmera Tapo no PC/NAS/NVR por RTSP/ONVIF (portas 554/2020, conta da câmera).
- ONVIF — Profile S; biblioteca `onvif-zeep`.
- OpenCV — FaceDetectorYN (YuNet) e FaceRecognizerSF (SFace): https://docs.opencv.org/ e https://github.com/opencv/opencv_zoo
- MediaPipe Gesture Recognizer: https://ai.google.dev/edge/mediapipe/
- NIST FRVT — avaliações de reconhecimento facial e efeitos demográficos.
- LGPD — Lei 13.709/2018, arts. 5º, 7º, 11 (dados sensíveis).
- Códigos: `aula-tapo/codigo/` e `aula-tapo/python/`.

# Anexo A — Scripts de referência

**rostos.py**

```
"""Reconhecimento facial com OpenCV (YuNet detecta, SFace reconhece) - modulo usado por
cadastrar_rosto.py e tapo_gestos_faces.py.


Como funciona:
  1. YuNet (rede neural leve) encontra rostos na imagem e 5 pontos (olhos, nariz, boca).
  2. SFace alinha o rosto por esses pontos e gera um "vetor de caracteristicas" de 128 numeros
(embedding).
  3. Reconhecer = comparar o vetor do rosto novo com os vetores cadastrados (similaridade de
cosseno).
     > 0,363 = mesma pessoa (limiar recomendado pelo OpenCV). Nenhuma imagem e guardada: so os
vetores.
Modelos (python/modelos/): face_detection_yunet_2023mar.onnx  e
face_recognition_sface_2021dec.onnx
"""
import os, cv2, numpy as np


PASTA = os.path.dirname(os.path.abspath(__file__))
MODELOS = os.path.join(PASTA, "modelos")
BANCO = os.path.join(PASTA, "rostos_cadastrados.npz")
LIMIAR_COS = 0.363


class Reconhecedor:
    def __init__(self, tamanho=(320, 320)):
        self.det = cv2.FaceDetectorYN.create(os.path.join(MODELOS,
"face_detection_yunet_2023mar.onnx"), "", tamanho, 0.8, 0.3, 5000)
        self.rec = cv2.FaceRecognizerSF.create(os.path.join(MODELOS,
"face_recognition_sface_2021dec.onnx"), "")
        self.nomes, self.vetores = [], np.zeros((0, 128), np.float32)
        self.carregar()
    def carregar(self):
        if os.path.exists(BANCO):
            d = np.load(BANCO, allow_pickle=True); self.nomes = list(d["nomes"]); self.vetores =
d["vetores"].astype(np.float32)
    def salvar(self): np.savez(BANCO, nomes=np.array(self.nomes), vetores=self.vetores)
    def detectar(self, img):
        """devolve lista de rostos: cada um e um vetor [x, y, w, h, 5 pontos..., confianca]"""
        self.det.setInputSize((img.shape[1], img.shape[0]))
        _, rostos = self.det.detect(img)
        return [] if rostos is None else rostos
    def vetor(self, img, rosto):
        alinhado = self.rec.alignCrop(img, rosto)
        return self.rec.feature(alinhado)[0]
    def cadastrar(self, nome, img, rosto):
        self.nomes.append(nome); self.vetores = np.vstack([self.vetores, self.vetor(img, rosto)
[None, :]]); self.salvar()
    def quem(self, img, rosto):
        """devolve (nome ou 'desconhecido', similaridade)"""
        if len(self.nomes) == 0: return "desconhecido", 0.0
        v = self.vetor(img, rosto)
        v = v / (np.linalg.norm(v) + 1e-9); base = self.vetores / (np.linalg.norm(self.vetores,
axis=1, keepdims=True) + 1e-9)
        sims = base @ v; i = int(np.argmax(sims))
        return (self.nomes[i], float(sims[i])) if sims[i] > LIMIAR_COS else ("desconhecido",
float(sims[i]))
    def desenhar(self, img, rosto, texto, cor):
        x, y, w, h = [int(v) for v in rosto[:4]]
        cv2.rectangle(img, (x, y), (x + w, y + h), cor, 2)
        cv2.putText(img, texto, (x, max(20, y - 8)), cv2.FONT_HERSHEY_SIMPLEX, 0.7, cor, 2)
```

**tapo_gestos_faces.py**

```
"""Projeto final: a camera Tapo ve -> o PC reconhece gestos (MediaPipe) e rostos (OpenCV SFace)
-> o ESP32 age.


Uso:
    python tapo_gestos_faces.py rtsp://u:s@192.168.1.50:554/stream2 --esp luzes.local
    python tapo_gestos_faces.py --webcam --esp 192.168.1.80
    python tapo_gestos_faces.py --webcam --sem-esp            # so reconhece
Regras (mude em REGRAS):
    gesto Open_Palm  -> liga a luz        Closed_Fist -> desliga        Thumb_Up -> LED verde
Victory -> LED azul
```

```
    rosto CONHECIDO  -> "bemvindo" (2 bipes + verde) uma vez a cada 60 s por pessoa
    rosto DESCONHECIDO por 3 s seguidos -> alerta "intruso" (sirene + vermelho) e foto salva em
alertas/
    ninguem -> nada.  Tecla q sai.
Modelos: modelos/gesture_recognizer.task (copie da aula ESP32-CAM) + modelos/*.onnx (rostos).
"""
import sys, os, time, threading, cv2, numpy as np, requests
import mediapipe as mp
from mediapipe.tasks.python import vision, BaseOptions
from rostos import Reconhecedor


args = sys.argv[1:]
FONTE = args[0] if args and not args[0].startswith("--") else "--webcam"
ESP = args[args.index("--esp") + 1] if "--esp" in args else "luzes.local"
ENVIAR = "--sem-esp" not in args
PASTA = os.path.dirname(os.path.abspath(__file__))
MODELO_GESTOS = os.path.join(PASTA, "modelos", "gesture_recognizer.task")
REGRAS = {"Open_Palm": "/luz?on=1", "Closed_Fist": "/luz?on=0", "Thumb_Up": "/led?cor=verde",
"Victory": "/led?cor=azul", "Thumb_Down": "/alerta?tipo=off"}
TEMPO_INTRUSO = 3.0; COOLDOWN_BEMVINDO = 60; COOLDOWN_INTRUSO = 30
os.makedirs(os.path.join(PASTA, "alertas"), exist_ok=True)


def enviar(rota):
    if not ENVIAR: return
    def _go():
        try: requests.get(f"http://{ESP}{rota}", timeout=2); print("ESP32 <-", rota)
        except requests.RequestException: print("(ESP32 nao respondeu em", ESP, ")")
    threading.Thread(target=_go, daemon=True).start()


gestos =
vision.GestureRecognizer.create_from_options(vision.GestureRecognizerOptions(base_options=BaseOpt
ions(model_asset_path=MODELO_GESTOS), num_hands=1))
rec = Reconhecedor()
os.environ.setdefault("OPENCV_FFMPEG_CAPTURE_OPTIONS", "rtsp_transport;tcp")
cap = cv2.VideoCapture(0) if FONTE == "--webcam" else cv2.VideoCapture(FONTE, cv2.CAP_FFMPEG)
if not cap.isOpened(): print("nao abriu", FONTE); sys.exit(1)
print(f"fonte {FONTE} | ESP32 {ESP} | pessoas cadastradas: {sorted(set(rec.nomes))}")


hist_gesto, ultimo_gesto_enviado = [], None
desconhecido_desde = None; ultimo_bemvindo = {}; ultimo_intruso = 0
while True:
    ok, q = cap.read()
    if not ok: time.sleep(0.5); continue
    agora = time.time()
    # ---- gestos ----
    res = gestos.recognize(mp.Image(image_format=mp.ImageFormat.SRGB, data=cv2.cvtColor(q,
cv2.COLOR_BGR2RGB)))
    gesto = res.gestures[0][0].category_name if res.gestures else "None"
    hist_gesto = (hist_gesto + [gesto])[-5:]
    if hist_gesto.count(gesto) >= 4 and gesto in REGRAS and gesto != ultimo_gesto_enviado:
        enviar(REGRAS[gesto]); ultimo_gesto_enviado = gesto
    if gesto == "None": ultimo_gesto_enviado = None
    # ---- rostos ----
    rostos = rec.detectar(q); ha_desconhecido = False
    for f in rostos:
        nome, sim = rec.quem(q, f)
        if nome == "desconhecido":
            ha_desconhecido = True; rec.desenhar(q, f, f"desconhecido ({sim:.2f})", (0, 0, 255))
        else:
            rec.desenhar(q, f, f"{nome} ({sim:.2f})", (0, 255, 0))
            if agora - ultimo_bemvindo.get(nome, 0) > COOLDOWN_BEMVINDO: ultimo_bemvindo[nome] =
agora; enviar("/alerta?tipo=bemvindo"); enviar("/luz?on=1")
    if ha_desconhecido:
        desconhecido_desde = desconhecido_desde or agora
        if agora - desconhecido_desde > TEMPO_INTRUSO and agora - ultimo_intruso >
COOLDOWN_INTRUSO:
            ultimo_intruso = agora; enviar("/alerta?tipo=intruso")
            foto = os.path.join(PASTA, "alertas", time.strftime("intruso_%Y%m%d_%H%M%S.jpg"));
cv2.imwrite(foto, q); print("foto salva", foto)
    else: desconhecido_desde = None
    cv2.putText(q, f"gesto: {gesto}   rostos: {len(rostos)}", (10, 28), cv2.FONT_HERSHEY_SIMPLEX,
0.8, (255, 255, 0), 2)
    cv2.imshow("Tapo + ESP32 (q sai)", q)
    if cv2.waitKey(1) & 0xFF == ord("q"): break
cap.release(); cv2.destroyAllWindows()
```